Level Up Coding · Follow publication

Member-only story

AI Isn’t Replacing Developers. It’s Doing Something Worse.

The replacement narrative is loud. The quiet one is already happening.

7 min read · May 11, 2026

Michael Lawrence Follow

Listen Share More

Generated by Midjourney

A few weeks ago I was reviewing a PR I’d put together with Claude Code. Standard refactor, splitting a chunky service class into something cleaner. The diff was maybe four hundred lines. I scanned it. Tests green. I approved it.

Two days later I was back in the same area for an unrelated reason and noticed something off. One of the methods I'd "refactored" had quietly lost a null check. Not catastrophically, the path it gated wasn't hot. But it was gone. I'd reviewed the diff. The tests had passed. I'd shipped it.

Twenty-five years of writing code, and the version of me from 2020 would have caught it in twelve seconds.

I sat with that longer than I want to admit. Not because of the bug. Bugs are bugs. I sat with it because of *how the bug got through*.

That's the piece I want to write about. Not the bug. The how.

The Wrong Worry

There's a conversation happening in our industry right now that I think is missing the point. The conversation is about replacement. Will AI replace developers? When? How many? Half the headlines you see are some flavor of that question, and half the comments under those headlines are some flavor of "no, it's just a tool" or "yes, you're cooked, learn to weld."

I don't want to spend much of this piece on that fight. I've shipped real code with AI. I work in a domain where it actually matters whether the code is right. I don't think AI is going to replace developers any time soon, and I don't think being smug about that is interesting either.

The replacement frame is loud. It's also the wrong worry.

The right worry is quieter, and it's already happening. It's been happening for a couple of years now, in the daily work of every developer who has adopted these tools seriously. It just doesn't have a clean headline attached to it.

Deformation

AI isn't replacing developers. It's deforming them.

I mean that word literally. *Deformation*: the process of changing the shape of something through applied pressure, often without the thing itself realizing it's happening. The developer doesn't disappear. The developer changes shape. Judgment, muscle memory, the ability to read code and feel that something is wrong before you can say why. Those don't go away in a dramatic event. They erode quietly, over months, while you're being told you're more productive than you've ever been.

That null check I missed? That was deformation talking. It wasn’t a knowledge gap. I know what null checks are. I’ve written ten thousand of them. The skill that failed wasn’t *knowing*. It was the daily reading muscle. The one that scans a diff like a detective and notices the absence of a thing that should be there.

That muscle gets built by reading a lot of code, slowly, with skin in the game. It gets maintained the same way. And for the last year and change, I’ve been reading less code that way. Not zero. Less.

The replacement narrative wants to know whether developers will exist in five years. I want to know what *kind* of developers will exist. Because the pipeline that produces good ones is being rerouted right now, and almost nobody is naming it.

Three Shapes

Deformation is showing up in three places. They look related. They’re actually different problems, and they’re going to need different answers.

Juniors. The first three to five years of a developer’s career used to be a slow, painful argument with the machine. You wrote code that didn’t work. You read other people’s code that you didn’t understand. You debugged things at 11 PM while questioning your career choices. That suffering wasn’t a bug in the path, it *was* the path. It built the mental models, the instincts, the tolerance for ambiguity that separates a developer from someone who can produce code. AI removes the suffering at exactly the developmental stage where suffering is the curriculum. Juniors entering the field right now can answer almost any concrete question they face by asking a model. Which means they may never sit with the question long enough for the answer to teach them anything. They don’t atrophy. They just never form.

Mid-levels. The five-to-eight-year crowd are in a worse spot than people are giving them credit for. They came up the right way. They built the muscles. Now they’re being told they’re slow if they don’t use AI, that their habits are outdated, that the careful path they took to seniorhood was *actually* the path of someone who didn’t get the memo. They’re caught between two formation models: the one that produced them and is now being publicly devalued, and the new one that’s promising to skip the part of the journey they just finished walking. There’s no clean way to play this from the inside. They’re either branded as resistant, or they capitulate and watch the muscles they spent eight years building start to soften.

Seniors. This is where the confession at the top comes in. Skills don’t atrophy because you stopped wanting them. They atrophy because the daily reps stopped happening. The senior developer who’s been heavily AI-assisted for two years has done less close reading of code, less from-scratch problem solving, less of the slow methodical debugging that used to make up their day. The skills are still there in some sense. But the edge is slipping in ways that are hard to feel from the inside, until the day you ship a refactor with a missing null check and have to sit with what that means.

Three different problems. One pattern underneath them.

Friction Was the Lesson

Most of what people called “the hard part of programming” was friction. Looking up the API. Reading the codebase to understand the existing patterns. Writing the test before you knew what you were testing. Sitting with a bug long enough to recognize it as a member of a category of bugs you’re going to see a hundred more times in your career.

Friction is where understanding lives. Not as a metaphor. The act of writing something by hand is what builds the mental model of the system you’re writing it in. The act of debugging slowly is what teaches you the shape of the errors that will bite you again in five years. The act of reading other people’s code carefully is what calibrates your sense of what good and bad code feel like under your eyes.

AI doesn’t slow developers down. That’s the whole point of it. But it removes the friction at exactly the moments that used to teach.

You don’t lose what you’ve already learned. You stop adding to it. And if you’re not paying attention, the pieces you stopped using start to dim.

That’s the deformation. That’s all it is. Not catastrophic. Not loud. Just the daily reps disappearing one at a time until the version of you that would have caught it in twelve seconds isn’t quite there anymore.

What This Is

I’m not writing this series to tell you to stop using AI. I use AI. I shipped a real iOS app with it last quarter. I’ll keep using it tomorrow.

I’m writing it because the conditions that produced the developers you respect. The ones you’d trust on a 3 AM page, the ones who can read a stack trace and tell you what happened upstream three services ago, those conditions are changing. The pipeline that turns a confused twenty-two-year-old into a person you’d hire is being quietly altered. The mid-level’s careful path is being publicly devalued. The senior’s daily muscle work is being outsourced. And almost nobody in the industry is naming any of this clearly, because most of the people writing about AI either need it to be the future, or need it to be a fad.

It’s neither. It’s a tool with real capabilities and a real cost, and the cost isn’t being paid by the tool.

It’s being paid by the people who use it.

Over the next three months I’m going to write five more pieces on this. One on what’s happening to juniors, and why the formation problem is the one I’m most worried about. One on the mid-level squeeze, the most painful spot to be in right now and the least talked about. One on senior atrophy, written from a 25-year vantage point: what you can lose, what you can’t afford to lose, and how to tell the difference. One on how to actually use these tools without deforming, which is the part nobody seems to want to write because it requires admitting both that the tools are useful *and* that they have a real cost. And finally one on what I’d tell a junior developer in 2026 if they asked me where to put their attention for the next decade.

I don’t think AI is the end of developers. I think it’s the end of one kind of formation, and the start of another that nobody has clearly defined.

If you’re a developer right now, you’re in a generation that’s going to look back at this period and either say “I noticed what was happening” or “I didn’t, and it cost me something I can’t quite name.”

The first step is naming it.

The second step is figuring out what to do about it.

Thanks for reading my articles! If you enjoyed this article, please clap many times. It would mean a lot to me and help other people see the story.

Follow me on X!

You can also find me at The Stoic Coder! Hope to see you over there.

Software Development | Agentic Ai | AI | Software Engineering | Careers

Follow

Published in Level Up Coding

337K followers · Last published 6 days ago

Coding tutorials and news. The developer homepage gitconnected.com && skilled.dev && levelup.dev

Follow

Written by Michael Lawrence

1.94K followers · 117 following

Michael Lawrence writes The Stoic Coder, a Substack on applying Stoic philosophy to the working developer's life. 25 years in the industry.